

Movie Recommendation System: A Comparative Study of Content-Based and Collaborative Filtering Approaches

1. Introduction

From recommending the next film a viewer should watch on Netflix to the products Amazon users are most likely to purchase, recommendation systems have grown into a central feature of digital platforms today. The basic task these systems attempt to solve is easy to explain but hard to do: What is the next item that should be presented to a certain user, based on a list of items and a history of their interaction with them? There are two classical families of algorithms that approach the problem differently. Content-based filtering recommends items based on their intrinsic properties (tags, descriptions, genres) that are similar to items that the user has already liked. The collaborative filtering approach, on the other hand, does not consider any item attributes, and simply relies on the aggregate behavior of users (recommended items are those that other users with similar rating histories have liked). In this report, the two methods are fully implemented and compared, and an extra matrix factorization model and an item-based collaborative variant are introduced in a synthetic movie dataset. The objective is not only to create working recommenders, but to know when which works and where it doesn't, and what a real company should do with these insights.

2. Dataset Description

To guarantee that the project was original, the class demo dataset was only provided as a reference, and a new, original movie dataset is created synthetically. Three components of the data set are logically related. The first is a collection of 75 movies that are distributed evenly across six categories: Action, Comedy, Drama, Sci-Fi, Horror, and Romance, each with an individual movie name, a movie genre label, five descriptive tags, a movie release year, a duration (in minutes), an average rating, and a movie description. Second, a population of 320 synthetic users split into four covert "taste clusters": users who like Action/Sci-Fi, users who like Drama/Romance, users who like Comedy/Horror, and users who like a wide range of tastes. Third, a set of approximately 3,500 ratings, where each user rated about 11 movies, and the probabilities were weighted according to their genre preferences (plus Gaussian noise to make the ratings behave realistically). There are also timestamps (within 2024) for each rating. This user-item matrix is approximately 87% sparse, reflecting real-world recommendation scenarios and making the evaluation more meaningful.

3. Data Preparation and Exploratory Data Analysis (EDA)

The data was loaded into pandas DataFrames and explored prior to modelling. The genre distribution is fairly even (12-13 movies in each genre), the average number of ratings per user is about 11, and the average rating value is about 3.9 (on a 1-5 scale) which is slightly positively skewed, typical of user-generated ratings. The user-item matrix was formed by performing a pivot on the data, leaving NAs for the missing values, thus keeping the distinction between "rated a 1" and "did not rate". The observed lack of density of ~87% is a very demanding environment for collaborative models. Three exploratory charts were created: Movies for each genre, a histogram for each rating per user, and a bar chart for the distribution of rating values. These graphs showed that there are no apparent anomalies in the synthetic data and that the structure is as expected.

4. Content-Based Filtering

In the content-based recommender, each movie is considered as a bag of words. The tags and description of each movie were combined into a single text field and TF-IDF (Term Frequency–Inverse Document Frequency) was used to convert it to a sparse vector. This produces a $75 \times N$ matrix where each row is a movie's numerical fingerprint emphasizing rare, discriminative words. Each movie is then compared with every other movie using cosine similarity, resulting in a 75×75 movie similarity matrix. For a user, the similarity score of each movie that they rated at 4 or higher was multiplied by the ratings given and then the scores were averaged to create a ranked list of movies similar to the ones the user has already enjoyed. A user who gave a high rating to many of the Action movies was recommended to watch more Action movies, with a few Sci-Fi movies thrown in, as expected. The biggest advantage of this method is that it is applicable even for a new movie that has no ratings; recommendations are made solely based on the movie's text.

5. Collaborative Filtering — User-Based

The user-based collaborative model assumes that users who have rated movies in the same manner will be interested in viewing the same type of movies. A user $\times$ movie rating matrix was built, with missing ratings replaced by ZERO and cosine similarity was calculated between all user pairs, resulting in a 320×320 user-similarity matrix. The 20 most similar users were found, and a similarity weighted average was used to predict a rating. The model is best able to perform when user's rating patterns overlap, as in the synthetic data, where users in the same

taste cluster get similar recommendations. Its drawbacks, however, are evident: it is not able to make recommendations to a new user who has no preferences, and it suffers from a high rate of error as the sparseness of the ratings increases, since similarity between users becomes noisy when few movies are co-rated.

6. Collaborative Filtering – Item Based (Additional Variation)

An extended modification was made as stipulated in the assignment brief – item-based collaborative filtering. This approach is not only based on comparing users, but also on movies, which are compared to other movies by the users who rate them similarly. The user-item matrix was only transposed, and cos-similarity calculated between movies creating another 75×75 matrix — one based on rating behaviour instead of content. The similarity scores of the movies that a user rated were weighted by their ratings and then added together to recommend movies for him/her. In production systems, item-based filtering is usually more stable than user-based filtering because the similarity of the items is much less volatile than that of the users (the audience of a movie is not going to change overnight); however, with user-based filtering, the similarity of users is changing with every new rating. To include this variation adds to the comparison and directly fulfills the assignment which asks for an additional modelling attempt.

7. Matrix Factorization — Truncated SVD

The final model uses Truncated Singular Value Decomposition on the mean centered user-item matrix. The data was split 80/20 into training and test sets. These missing values in the training matrix were imputed with the global mean rating prior to the centering, to avoid being influenced by "unobserved" cells to factorize toward zero. The SVD transformed the matrix into 20 latent dimensions that represent the hidden taste factors of interest, like “prefers slow emotional films” or “prefers fast action-driven plots”, which are not so apparent from raw similarity. The predicted ratings were reconstructed by multiplying the user-factor and item-factor matrices and adding back the global mean and clipping to the range [1, 5]. This model achieved the highest number of correct answers on the test set that was not used to train the model.

8. Evaluation and Model Comparison

Two metrics were used: In a collaborative model, Root Mean Squared Error (RMSE) was used to compare the accuracy of each model prediction on the 20% held-out test set. Naive baseline (always predicting the global mean rating) set the baseline. The baseline model had an RMSE

around 0.97, the user-based collaborative model around 0.93 and the SVD matrix factorization model the best score, around 0.90 on the test set. Due to the fact the content-based filtering does not predict a single rating, it was evaluated qualitatively by examining the recommendations for different users and verifying if the recommendations correspond to the rated genre preferences of each user, by checking the recommendations for multiple users. The RMSE values were compared by a bar chart which showed that SVD performs better than the baseline CF and user-based CF. A scatter plot of predicted versus actual was shown for the SVD example, and a reasonable clustering around the diagonal was seen, with the compression of the predictions toward the mean as is well known with matrix factorization models on sparse data.

9. Cold-Start Demonstration

The cold start problem was shown experimentally. A new film ("The Brand-New Signal", Sci-Fi, 2025) was added and has no rating. Content-based filtering was able to get 5 similar Sci-Fi recommendations, as it requires just the movie text. However, Collaborative filtering could not rate the new movie: the model does not have any rating history for the new movie; it can't be placed in the user-item matrix. On the other hand, if the user hasn't rated any content and the brand-new user doesn't have ratings, neither collaborative nor content based could help, and content-based filtering can only assist after the user has interacted with at least one content. This asymmetry (content-based solving item cold-start, collaborative cold-start user), is the main practical trade-off which drives hybrid systems.

10. Limitations

There are three caveats in this study. First, the data is artificial. The RMSE reported values were intentionally designed to be below what these same models would obtain using noisy real-world data. Secondly, the spatial density is low. The matrix is sparse, having only ~11 ratings for each user of 75 movies. This means that similarity-based recommendations can't be as accurate as they can be, and users that rated the movies under 5 will only be given pretty irrelevant collaborative recommendations. Third, catalog is compact. The number of movies is much less than what is available in commercial systems, and so, the similarity scores in this project are more prone to noise and less stable than when scaled up. The general order of magnitude of the models (SVD > user-based CF > baseline) is a strong and consistent result, despite these limitations.

11. Proposed Improvement — Hybrid Recommender

The logical sequel is a hybrid recommender. The basic concept is to use a weighted score by combining the best elements of both families: $\text{collaborative_score} = \text{final_score} - \alpha \times \text{content_score}$ where α is fine-tuned on a validation set. If α is large, the system will act more like a content-based recommender, which is useful in the following cases: new user, new items. When α is small, it acts more like a collaborative recommender, which is more useful for users who have extensive usage record. It is easy to demonstrate that a basic production process is to set $\alpha = 1$ for new users and slowly lower α as the users' rating history increases. Other hybrid implementations include switching (where collaborative data is not enough, then use content-based), cascade (where a) content-based shortlists and b) collaborative ranks) and feature augmentation (where feed SVD latent factors into the content-based vectorizer). All of these would overcome the cold start restrictions described in Section 9.

12. Business Implication

If the company is operating this catalog, it is clear that a hybrid system is recommended from the beginning of operation, but it should be weighed dynamically according to user maturity. This means that, for new visitors, they should see recommendations based on content from the movies they click or rate right away, transforming new visitors into engaged users. The more users who return, the more they have rated products and the more they have seen, the more they should move towards the collaborative model (SVD), which gave the lowest RMSE and the most personalized results. A small portion of traffic (say 5-10%) should be allocated to exploration – showing slightly less confident pages to gain more understanding on user preferences which will help the model get better with time. Last but not least, the company should not only rely on RMSE as a metric for optimizing precision@k, diversity, novelty and long-term user retention are all equally important for the success of the business, and are the logical successors of this project.

13. Conclusion

The movie recommender project was designed to construct and evaluate three types of movie recommender systems: content-based, collaborative (user-based and item-based) and matrix factorization on a synthetic movie dataset containing 75 movies, 320 users, and ~3,500 ratings. The empirical results reveal that the SVD matrix factorization approach was the one with the smallest RMSE, as compared to the naive baseline model and user-based collaborative filtering.

Content-based filtering was not only not reducible to RMSE but was successful in the cold-start case where it was able to suggest movies that were similar to a completely new movie that collaborative filtering could not. The key message is that there is no one model that is best in every context: content-based works well with new items; collaborative works well with established users; matrix factorization works well with discovering latent taste patterns. This is why it is most practical to use a well-designed hybrid system and why this project would have continued to follow this route.

Github Repository

<https://github.com/sourav1903078/movie-recommendation-system>